

CAD-HW03-exe2

Hanie Vatani

14000122668035

برای طراحی یک شیفت رجیستر 64-bit که دارای چهار ورودی و یک خروجی است باید مراحل زیر را طی کنیم. ورودی های آن clk، 3-bit Command، 640bit DataIn، و خروجی 64-bit DataOut، است. ورودی Reset به صورت آسنکرون باید رجیستر را صفر کن. سپس برای نوشتن کد از دستورات گفته شده که در صورت سؤال درج شده کمک می گیریم.

کد برنامه :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity exe2 is
    port (
        Reset, clk: in std_logic;
        Command: in std_logic_vector(2 downto 0);
        DataIn: in std_logic_vector(63 downto 0);
        DataOut: out std_logic_vector(63 downto 0)
    );
end exe2;

architecture exe2 of exe2 is
    signal reg: std_logic_vector(63 downto 0);
begin
    process (clk, Reset, Command, DataIn)
    begin
        if (Reset = '1') then
            reg <= (others => '0');
        elsif (clk'event and clk = '1') then
            case Command is
                when "001" =>
                    reg <= DataIn;
                when "010" =>
                    reg <= reg(62 downto 0) & '0';
                when "011" =>
                    reg <= '0' & reg(63 downto 1);
                when "101" =>
                    reg <= reg(62 downto 0) & reg(63);
                when "110" =>
                    reg <= reg(0) & reg(63 downto 1);
                when others =>
                    reg <= reg;
            end case;
        end if;
    end process;
    DataOut <= reg;
end exe2;
```

همانطور که در صورت سؤال گفته شده:

- زمانی که Command برابر 001 باشد آنگاه ورودی را در رجیستر ثبت می‌کنیم.
 - زمانی که Command برابر 010 باشد آنگاه داده‌ی درون رجیستر را شیفت به چپ می‌دهیم.
 - زمانی که Command برابر 011 باشد آنگاه داده‌ی درون رجیستر را شیفت به راست می‌دهیم.
 - زمانی که Command برابر 101 باشد آنگاه داده‌ی درون رجیستر را شیفت به چپ حلقوی می‌دهیم.
 - زمانی که Command برابر 110 باشد آنگاه داده‌ی درون رجیستر را شیفت به راست حلقوی می‌دهیم.
- اگر Command هر چیزی جز این حالات باشد آنگاه داده‌ی درون رجیستر تغییر نمی‌کند.

سپس کد را compile کرده و فایل test bench برای شبیه سازی می سازیم. و به آن کد زیر را اضافه می کنیم:

```
DataIn <= "1101100101010000111001110110010110110110011001100101011010101";
Reset <= '1', '0' after 5ns;
clk <= '0', '1' after 3ns, '0' after 10ns, '1' after 15ns, '0' after 20ns, '1' after 25ns, '0' after 30ns, '1' after 35ns,
      '0' after 40ns, '1' after 45ns, '0' after 50ns, '1' after 55ns, '0' after 60ns, '1' after 65ns, '0' after 70ns, '1' after 75ns;
Command <= "0000", "0011" after 10 ns, "0101" after 20ns, "0111" after 30ns, "1000" after 40ns, "1011" after 50ns, "1100" after 60ns, "1111" after 70ns;
```

بعد از compile کردن فایل test bench آن را شبیه سازی می کنیم. برای این کار فایل test bench را excute می کنیم. و سپس برنامه را run می کنیم.

نمونه ای از شبیه سازی ها:

Signal name	Value
rst Reset	1
clk	1
Command	000
DataIn	110110010101000101100111011001011011011001100110010101101101
DataOut	00
reg	00

The timing diagram displays several digital signals over a time period from 0 to 72 ns. The signals shown are:

- rst (Reset):** High from 8 ns to 72 ns.
- clk (Clock):** A periodic square wave with a period of approximately 4 ns.
- Command:** Constant at 000.
- DataIn:** A constant stream of bits: 110110010101000101100111011001011011011001100110010101101101.
- DataOut:** Constant at 00.
- reg (Register):** Constant at 00.

A red vertical line indicates a specific point in time at 6130 ps (6.13 ns).



